

LegalEase - AI-Powered Legal Document Generator

LegalEase - AI-Powered Legal Document Generator

Government Arts and science college-Manur-MSU142

Team leader:

Nithyakala p NM ID:23FAA3BBED5DBA0932A413567D171184

Team members: Esakki priya p NM ID:77199CD4EDE1276B9514C63C27537B04

Praba Nanthini N NM ID:3CC256046670873B4DD4E3FF32B713BA

PHASE 3: PROJECT DESIGN PHASE

1. Objective of Design Phase:

The objective of this phase is to design the complete system architecture, user interface, and data flow for the LegalEase application. This phase acts as a blueprint before actual coding begins.

2. System Architecture Design:

Our system follows a 3-Tier Architecture.

Tier 1 - Presentation Layer (Frontend):

Technology: Streamlit (Python Framework)

Responsibility: User interaction, collecting inputs like Party Names, Document Type, Terms, and displaying the generated document with edit option.

Tier 2 - Application Layer (Backend):

Technology: FastAPI (Python Framework)

Responsibility: Acts as a bridge between frontend and AI. It has two main API endpoints:

a) POST /generate - Receives user details and calls AI Core.

b) POST /download - Receives final content and converts it into DOCX/PDF for download.

It also handles formatting logic using python-docx and fpdf2 libraries.

Tier 3 - Intelligence Layer (AI Core):

Technology: Google Gemini 1.5 Pro API

Responsibility: The brain of the system. It receives a well-engineered prompt with user details and generates a professional legal document with clauses, sections, and legal language.

3. Data Flow Diagram (DFD):

Level 0 DFD (Context Level):

User -> Enters Details -> [LegalEase System] -> Generates Legal Document -> User

Level 1 DFD (Detailed Level):

Step 1.0 - Input Collection: Frontend collects doc_type and details dictionary.

Step 2.0 - Prompt Engineering: Backend creates a structured prompt: "You are a professional legal drafter... Generate {doc_type} with {details}..."

Step 3.0 - AI Generation: AI Core sends prompt to Gemini API and gets response.text.

Step 4.0 - Output Formatting: Backend formats content into Branded DOCX (with Header, Table, Footer) and PDF.

Step 5.0 - Delivery & Edit: Frontend shows content in an editable text area and provides download buttons.

4. User Interface (UI) Design:

Screen 1: Input Screen

- Top: Title "LegalEase - AI Legal Document Generator" with icon.
- Middle: Dropdown - Select Document Type [Employment Contract, NDA, Lease Agreement]
- TextFields: Party 1 Name, Party 2 Name, Role/Purpose
- Date Picker: Start Date
- Text Area: Additional Terms and Conditions
- Button: "Generate Document" (Blue Color)

Screen 2: Output Screen

- Success Message: "Document Generated Successfully!"
- Editable Text Area: Shows generated document (Height 500px, so user can scroll and edit)
- 3 Columns for Download:
 - Column 1: "Download as TXT" Button
 - Column 2: "Download as DOCX (Branded)" Button
 - Column 3: "Download as PDF" Button

5. Database Design:

No permanent database is used to ensure privacy. Data is handled in temporary memory (RAM) using Python dictionaries and BytesIO buffers for file generation. This makes the system secure and lightweight.

6. Security Design:

- API Key Security: Gemini API key is stored in .env file and not exposed to frontend.
- Data Privacy: No data is stored on server after session ends.
- CORS Enabled: Backend allows all origins for seamless frontend-backend connection during development.